

RAPPORT DE VEILLE TECHNOLOGIQUE BACK-END

Kanban Board API

Comparaison de NestJS, Symfony et Spring Boot

Technologie retenue : **NestJS**

Étudiant	Fallou Gaye
Projet	Veille technologique back-end
Application	API Kanban Board
Technologie choisie	NestJS

1. Introduction

Dans le cadre du projet de veille technologique back-end, l'objectif est de choisir une technologie adaptée au développement d'une API pour une application de type **Kanban Board**. Trois solutions sont étudiées : **NestJS**, **Symfony** et **Spring Boot**.

Cette veille permet de comprendre leurs principales caractéristiques avant de justifier le choix réalisé pour le projet. La comparaison porte notamment sur leur architecture, leur facilité de prise en main, leur maintenabilité, leur écosystème et les mécanismes disponibles pour construire une application sécurisée.

Pour notre réalisation, nous avons retenu **NestJS** et développé une API permettant de gérer l'authentification des utilisateurs ainsi que les listes et les cartes d'un tableau Kanban.

2. NestJS – Présentation, avantages et inconvénients

NestJS est un framework back-end exécuté sur Node.js et conçu principalement pour TypeScript, même s'il reste compatible avec JavaScript. Sa documentation officielle le présente comme un framework destiné à construire des applications serveur efficaces et évolutives. Il fournit une architecture structurée reposant notamment sur des modules, des contrôleurs et des providers, avec un système d'injection de dépendances.

Cette organisation constitue un avantage important pour séparer les responsabilités et conserver un code lisible lorsque l'application grandit. NestJS propose aussi des mécanismes utiles pour une API : validation, Pipes, Guards, authentification, autorisation et intégration OpenAPI/Swagger.

Il reste indépendant de la base de données et peut être associé à Prisma, TypeORM ou d'autres outils. Dans notre projet, cette souplesse permet d'utiliser **Prisma avec PostgreSQL**.

En revanche, NestJS demande un temps d'apprentissage. Les décorateurs, l'injection de dépendances, les DTO, les Guards et l'organisation en modules représentent plusieurs concepts à assimiler. Pour une API très petite, cette structure peut paraître plus lourde qu'une application Express minimale. Enfin, l'écosystème Node.js implique de suivre les versions et la compatibilité des dépendances utilisées.

3. Symfony – Présentation, avantages et inconvénients

Symfony est un framework PHP destiné au développement d'applications web et d'API structurées. Sa documentation officielle présente une architecture reposant notamment sur les requêtes et réponses HTTP, le noyau, les services, l'injection de dépendances, les événements, les contrats et les bundles.

Le framework dispose aussi de nombreux composants qui peuvent être utilisés dans une application Symfony complète ou de manière plus indépendante. Cette modularité représente un avantage pour construire une application progressivement et réutiliser uniquement les briques nécessaires.

Symfony fournit des outils documentés pour les bases de données, la validation, les tests, la sérialisation, les files de messages et la sécurité. Son *SecurityBundle* prend notamment en charge l'authentification et l'autorisation, tandis que le framework propose des mécanismes liés aux mots de passe et à la protection CSRF. Son écosystème mature et sa documentation constituent donc des points forts pour la maintenance d'un projet.

En contrepartie, Symfony demande de connaître PHP moderne et plusieurs conventions propres au framework. Son système de services, sa configuration et la richesse de ses composants peuvent demander du temps avant d'être maîtrisés. Pour une API très simple, cette richesse fonctionnelle peut également sembler plus importante que les besoins immédiats du projet.

4. Spring Boot – Présentation, avantages et inconvénients

Spring Boot appartient à l'écosystème Spring et permet de développer des applications back-end autonomes, généralement en Java. Son objectif est de simplifier la création d'applications Spring prêtes pour la production.

La documentation officielle met notamment en avant l'auto-configuration, les dépendances *starter*, les serveurs embarqués et différentes fonctionnalités destinées à la production, comme les métriques, les contrôles de santé et la configuration externalisée.

L'auto-configuration adapte une partie de la configuration de l'application aux dépendances présentes, tout en laissant au développeur la possibilité de remplacer les choix automatiques. Spring Boot bénéficie également de l'écosystème Spring pour le web, l'accès aux données et la sécurité. *Spring Security* peut ainsi être intégré pour gérer l'authentification et l'autorisation.

Cette richesse constitue un avantage pour des applications importantes et des architectures destinées à évoluer. En revanche, l'écosystème Spring est vaste et sa prise en main peut demander du temps. Il faut comprendre les annotations, l'injection de dépendances, l'auto-configuration et les nombreuses briques disponibles.

Pour une petite API, cette richesse peut dépasser les besoins immédiats. Le développement Java et la configuration de l'environnement peuvent aussi paraître plus lourds à un développeur déjà habitué à l'écosystème JavaScript/TypeScript.

5. Justification du choix de NestJS

Pour notre API Kanban Board, nous avons retenu **NestJS**. Ce choix vient d'abord de notre volonté de travailler avec Node.js et TypeScript tout en utilisant une architecture plus structurée qu'une application Node.js minimale.

NestJS nous permet de séparer les responsabilités entre **modules**, **contrôleurs** et **services**. Cette organisation correspond bien à notre application, composée de plusieurs domaines : utilisateurs, authentification, listes et cartes.

Le projet nous a également permis d'utiliser concrètement plusieurs mécanismes du framework. Les **DTO** et le **ValidationPipe** contrôlent les données reçues par l'API, tandis qu'un **Guard** protège les routes nécessitant une authentification.

Nous avons mis en place des jetons **JWT**, des règles d'autorisation et des contrôles de propriété afin qu'un utilisateur ne puisse pas modifier les listes ou cartes d'un autre utilisateur. NestJS s'intègre aussi correctement avec les outils retenus : **Prisma** assure l'accès à PostgreSQL et **Swagger** documente les endpoints sous `/api`.

Le choix n'a donc pas été fait uniquement sur la popularité du framework. Il correspond à notre environnement TypeScript, aux besoins fonctionnels et de sécurité du Kanban Board, ainsi qu'à notre objectif d'apprendre une organisation modulaire et maintenable réutilisable dans de futurs projets back-end.

6. Sources de veille

Les informations techniques ont été vérifiées principalement à partir des documentations officielles des technologies étudiées.

— **NestJS – Documentation officielle**

<https://docs.nestjs.com/>

— **NestJS – Modules**

<https://docs.nestjs.com/modules>

— **NestJS – Database / Prisma et autres intégrations**

<https://docs.nestjs.com/techniques/database>

— **Symfony – Documentation officielle**

<https://symfony.com/doc/current/>

— **Symfony – Components**

<https://symfony.com/doc/current/components/>

— **Symfony – Security**

<https://symfony.com/doc/current/security.html>

— **Spring Boot – Documentation officielle**

<https://docs.spring.io/spring-boot/>

— **Spring Boot – Auto-configuration**

<https://docs.spring.io/spring-boot/reference/using/auto-configuration.html>

— **Spring Boot – Spring Security**

<https://docs.spring.io/spring-boot/reference/web/spring-security.html>

Les caractéristiques générales des frameworks ont été étudiées à partir de leurs documentations officielles. La justification du choix de NestJS repose également sur l'expérience acquise pendant le développement de l'API Kanban Board.